

Article

Exploration- and Exploitation-Driven Deep Deterministic Policy Gradient for Active SLAM in Unknown Indoor Environments

Shengmin Zhao  and Seung-Hoon Hwang * 

Division of Electronics and Electrical Engineering, Dongguk University, Seoul 04620, Republic of Korea; shengmin.zhao@dgu.ac.kr

* Correspondence: shwang@dongguk.edu; Tel.: +82-2-2260-3994

Abstract: This study proposes a solution for Active Simultaneous Localization and Mapping (Active SLAM) of robots in unknown indoor environments using a combination of Deep Deterministic Policy Gradient (DDPG) path planning and the Cartographer algorithm. To enhance the convergence speed of the DDPG network and minimize collisions with obstacles, we devised a unique reward function that integrates exploration and exploitation strategies. The exploration strategy allows the robot to achieve the shortest running time and movement trajectory, enabling efficient traversal of unmapped environments. Moreover, the exploitation strategy introduces active closed loops to enhance map accuracy. We conducted experiments using the simulation platform Gazebo to validate our proposed model. The experimental results demonstrate that our model surpasses other Active SLAM methods in exploring and mapping unknown environments, achieving significant grid completeness of 98.7%.

Keywords: active SLAM; DDPG; cartographer; exploration and exploitation strategies



Citation: Zhao, S.; Hwang, S.-H. Exploration- and Exploitation-Driven Deep Deterministic Policy Gradient for Active SLAM in Unknown Indoor Environments. *Electronics* **2024**, *13*, 999. <https://doi.org/10.3390/electronics13050999>

Academic Editor: Arturo de la Escalera Hueso

Received: 26 January 2024

Revised: 2 March 2024

Accepted: 4 March 2024

Published: 6 March 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Simultaneous Localization and Mapping (SLAM) [1] is a set of approaches in which a Robot Operating System (ROS) robot [2] autonomously localizes itself and simultaneously maps the environment while navigating through it. Localization and mapping are important components of SLAM. Localization involves estimating the robot's pose in relation to the surrounding environment. It also involves determining the robot's position and orientation in real time as it moves through space. Mapping focuses on creating an accurate representation of the environment using data from some sensors, like 2D LiDAR [3], inertial measurement units (IMUs) [4], and vision cameras [5]. These sensors provide perception information for the robot to comprehend its surroundings and construct a comprehensive map. Many techniques have been used to solve the SLAM problem, such as the extended Kalman filter (EKF) [6], scan-matching SLAM [7], and the particle filter [8]. These techniques have many limitations, such as complexity, data association ambiguity, and the quadratic cost of computing a joint map covariance matrix. Montemerlo et al. proposed the Fast SLAM method [9], which integrates the EKF and particle filter techniques. This method has limitations, such as sample impoverishment and particle depletion issues. To address these issues, Grisetti et al. proposed the grid mapping (Gmapping) approach to perform resampling operations selectively, effectively mitigating the problem of particle depletion [10]. However, the Gmapping algorithm can only maintain reliable accuracy in simple, low-feature indoor environments. The Gmapping method requires more particles in the mapping for larger and more complex environments, which results in increased calculation. Furthermore, the absence of closed-loop detection can cause cumulative errors, leading to deviations in mapping accuracy [11]. The Cartographer algorithm is an open-source Graph SLAM method developed by Google [12] that introduces a closed-loop detection link in the back-end module. Hess et al. proposed a real-time closed-loop detection method based on the Cartographer algorithm which employs the branch and bound method to conduct a systematic search and progressively refine the search space.

Active SLAM aims to address the challenge of optimal exploration in unknown environments by proposing a path planning strategy that generates actions to reduce map and pose uncertainty [13]. Alongside the localization and mapping, the path planning module plays a vital role in the Active SLAM framework. Traditional path planning algorithms, such as frontier-based exploration [14], rapidly exploring random tree (RRT) [15], the dynamic window approach (DWA) [16], A-star (A*) [17], and Dijkstra [18], were integrated into the robot with SLAM methods to implement Active SLAM. Carlone et al. proposed an application of Kullback–Leibler divergence to evaluate the particle filter SLAM posterior approximation [19]. This metric is applied to define the expected information from a policy, which allows the robot to decide between exploration and place-revisiting actions autonomously. Trivun et al. proposed a hybrid control-switching exploration method for Fast SLAM as the back end [20]. The method utilizes a frontier-based exploration technique with A* as the global planner and the DWA reactive algorithm as the local planner. Like this method, Michal et al. used A* to complete path planning but adopted the EKF-SLAM algorithm to complete the mapping [21]. However, their approach does not allow for searching for an optimal path and relies on greedy criteria for selecting the frontier to be visited. To reduce the Active SLAM computational complexity, Placed et al. treated the underlying graph as an estimation Open Karto SLAM problem and proposed a new method of computing the D-optimality criterion over the reduced weighted graph Laplacian matrix [22]. They then used the Dijkstra method to implement path planning. Suresh et al. focused on volume exploration in underwater environments by using multi-beam sonar. To ensure efficient path planning, the authors selected revisit actions based on pose uncertainty and sensor information gain, in combination with the RRT algorithm [23]. However, due to random characteristics, it was impossible to completely detect all unknown areas within a limited time.

Recently, researchers have applied reinforcement learning (RL) algorithms to Active SLAM tasks, such as Deep Q-Networks (DQN) [24], Advantage Actor–Critic (A2C) [25], and Dueling Double DQN (D3QN) [26]. Chen et al. trained DQN and A2C agents by incorporating the underlying SLAM pose graph through Graph Neural Networks (GNNs) [27]. In their study, they integrated DRL with exploration graphs to develop policies considering the map's uncertainty, surpassing the performance of random and nearest frontier agents. The authors demonstrated successful generalization results in state spaces with higher dimensions. However, it is important to note that the experiments conducted in that study utilized a simulation environment consisting of a basic grid world. This environment did not involve any obstacle avoidance tasks; therefore, there was no need to employ a SLAM method. Juna et al. presented an open-source Active SLAM framework that leverages the Graph SLAM system [28]. The framework uses fast utility computation by exploiting and implementing path planning through DQN. Castellanos et al. used the D3QN to explore the unknown environment and designed a reward function based on uncertainty to speed up network training [29]. The integration of the DQN algorithm into Active SLAM methods enhances the autonomy and intelligence of the robot in path planning. The above algorithms can only handle discrete action space problems. For example, when the robot is in a certain state (position), it can obtain different rewards by taking different actions (up, down, left, and right). However, it is important for robots to have a continuous movement and action space while exploring unknown areas. Additionally, most RL methods primarily concentrate on the exploration phase of robot SLAM and overlook the exploitation process.

An essential aspect of the exploitation is the ability to continuously optimize the representation of known areas of the environment, achieved by the robot actively exploiting previously explored areas. It improves mapping accuracy and reduces uncertainties about the robot's position and environmental characteristics. Additionally, the exploitation phase helps the robot identify and correct drawing errors or inaccuracies that arise during exploration. The system can adapt to environmental changes by revisiting and refining existing maps and ensuring a consistent and reliable representation. Table 1 summarizes the differences between our approach and other mentioned methods.

Table 1. Comparison between our approach and other Active SLAM methods.

Reference	Active SLAM		Reward Design	Focus on Exploration	Focus on Exploitation
	SLAM Method	Path Planning Method			
Carlone et al. [19]	Particle filter	Frontier-based exploration	-	✓	-
Trivun et al. [20]	Fast SLAM	A*, DWA ¹	-	✓	-
Michal et al. [21]	EKF SLAM	A*	-	✓	-
Placed et al. [22]	Open Karto	Dijkstra	-	✓	-
Suresh et al. [23]	Graph SLAM	RRT ²	-	✓	-
Chen et al. [27]	-	DQN ³ , GNN ⁴	T-opt	✓	-
Juna et al. [28]	Graph SLAM	DQN	D-opt	✓	-
Castellanos et al. [29]	Gmapping	D3QN ⁵	D-opt	✓	-
Ours	Cartographer	DDPG ⁶	EE ⁷	✓	✓

¹ Dynamic window approach. ² Rapidly exploring random tree. ³ Deep Q-Networks. ⁴ Graph Neural Network. ⁵ Dueling Double Deep Q-Networks. ⁶ Deep Deterministic Policy Gradient. ⁷ Exploration and Exploitation.

In this work, we propose an Active SLAM framework based on the DDPG and Cartographer for efficiently exploring and building maps based on robot-borne environmental sensor readings and information from SLAM algorithms, such as pose estimation and map integrity. The main contributions of this study are as follows:

- The exploration- and exploitation-driven reward function is proposed, which depends on the locations visited by the robot as well as its movement trajectories, allowing the strategy to be quickly learned and generalized to other complex environments without training.
- The DDPG algorithm is proposed for robot path planning based on the exploration- and exploitation-driven reward function. Compared to other methods, the DDPG excels in representing continuous motion and action spaces, enabling the robot to move at higher speeds and with smooth movement.
- The Active SLAM framework is proposed based on the DDPG, which simultaneously focus on robotic exploration and exploitation. In comparison to existing reinforcement learning-based methods, our proposed method enhances the completeness of SLAM maps.

This paper is organized as follows: Section 2 introduces the background information regarding reinforcement learning and SLAM, and Section 3 presents the proposed approach. Then, Section 4 details the experimental design and presents the results. Finally, Section 5 sets out the conclusions.

2. Background

2.1. Reinforcement Learning (RL)

Reinforcement learning (RL) is a machine learning (ML) branch that focuses on learning sequential decision making processes. The agent, also the decision maker, interacts with the environment to learn the best way to behave. This interaction can be represented as a Markov decision process (MDP) [30]. An MDP that combines Markov decision theory with dynamic programming (DP) is a straightforward and mathematically idealized form of RL problems. This framework allows for the concise representation of essential features of RL with only a few characteristics.

In general, an MDP is defined as a four-tuple $M = \{S, A, P, R\}$ [31], where S is a finite set of states that an agent can be while in the environment; A is a set of agent actions that an agent can take while in a state $s \in S$; P is a transition probability, since the actions are considered part of a probability distribution that represents distribution over the possible resulting state by taking a specific action while in a given state s ; R is a reward function, associated with a state transition by taking a specific action $R(S_t, a_t, S_{t+1})$. More specifically,

for a particular action $a \in A$ in a current state $s \in S$, the probability of transferring to a subsequent state s' occurring at time t can be expressed as follows:

$$P(s, a, s') = P(s'|s, a) = P(S_{t+1} = s' | S_t = s, A_t = a) \quad (1)$$

2.2. Deep Deterministic Policy Gradient (DDPG)

Value-based deep RL methods, such as DQN, and their variants have been developed. However, these methods are limited to handling discrete and low-dimensional action spaces. Continuous high-dimensional action spaces are common in Active SLAM tasks, particularly in path planning control tasks.

Lillicrap proposed the Deep Deterministic Policy Gradient (DDPG) algorithm [32], which utilizes a policy gradient method to optimize the policy directly. This approach is suitable for problems that involve a continuous action space. Unlike the random strategy represented by the probability distribution function at $a_t = \pi_\theta(s_t | \theta^\pi)$, DPG utilizes a deterministic policy function at $a_t = \mu(s_t | \theta^\mu)$. Additionally, it employs a convolutional neural network to simulate the policy and Q functions. It learns from the experience replay and target network in the DQN to stabilize the training and ensure high sample utilization efficiency. The Q network is gently updated by gradient ascent, where K samples from the experience pool are randomly selected. The loss function of the Q network is defined as follows:

$$L = \frac{1}{K} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2 \quad (2)$$

where $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$ indicates the expected value. The unbiased estimate of the policy network gradient is obtained as follows:

$$\nabla_{\theta^\mu} J \approx \frac{1}{K} \sum_i \nabla_a Q(s, a | \theta^Q) |_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) |_{s_i} \quad (3)$$

2.3. Cartographer SLAM Method

Cartographer can be viewed as consisting of two separate but related subsystems [12]. The first subsystem is the local SLAM, the frontend or local trajectory builder, as shown in Figure 1. Local SLAM incorporates a new scan into its ongoing submap construction by matching the scan with an initial guess obtained from the pose extrapolator. The pose extrapolator is designed to utilize sensor data from multiple sensors and the range finder to predict the optimal location for inserting the next scan into the submap. The primary objective of the pose extrapolator is to construct a sequence of submaps that maintain local consistency. Nonetheless, local SLAM may encounter drift over time.

While the local SLAM algorithm generates a sequence of submaps, a global optimization task, commonly known as ‘the optimization problem’ or ‘sparse pose adjustment’, runs in the background. Its purpose is to rearrange the submaps in such a way that they form a coherent global map. This optimization task is responsible for adjusting the currently constructed trajectory to align the submaps accurately, especially considering loop closures.

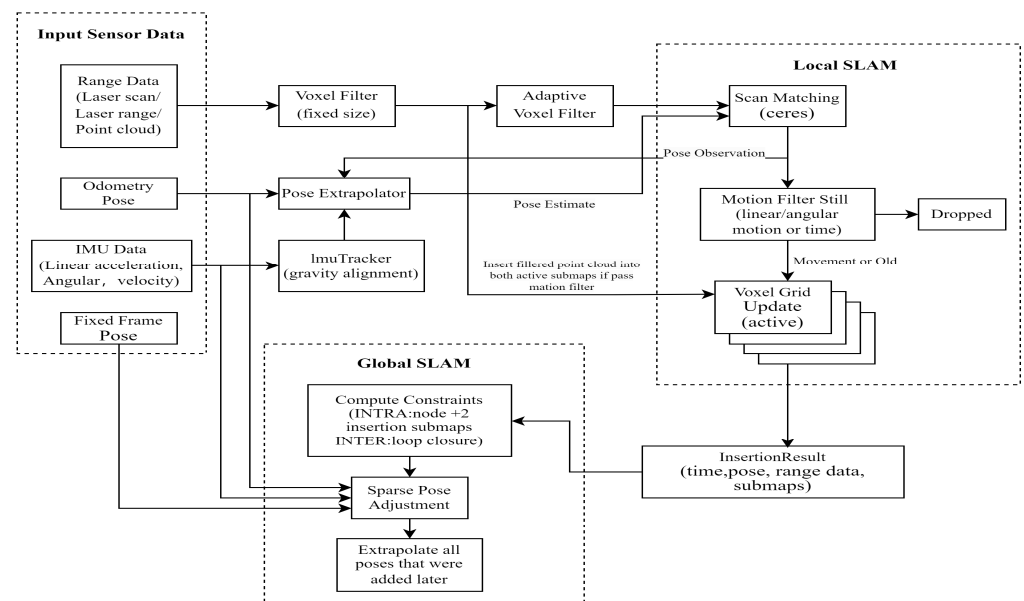


Figure 1. Flow chart of the Cartographer method.

3. Methodology

3.1. Proposed Framework for Active SLAM

An Active SLAM framework based on the DDPG is proposed in this study to explore unknown indoor environments and build maps autonomously. The combination of the DDPG for path planning and the Cartographer SLAM method is strengthened to construct maps in the most efficient way, as illustrated in Figure 2. In order to achieve this, two key aspects of reinforcement learning are focused on: reward functions and state spaces. For the reward function, an exploration- and exploitation-driven reward strategy is designed to encourage the robot to autonomously explore unknown areas and revisit traversed areas to improve map accuracy. The state space is further defined to enhance the mapping of indoor and unknown environments. The detailed steps of the framework are as follows:

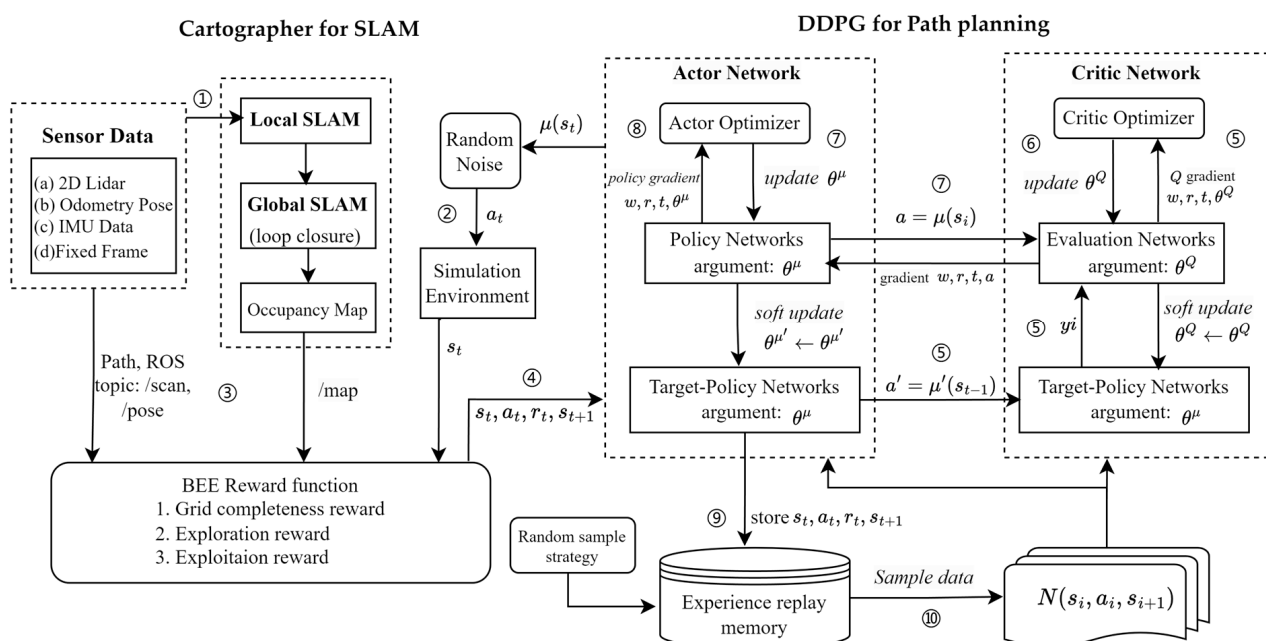


Figure 2. Algorithm flow chart of the DDPG-based Active SLAM framework.

1. The local SLAM module takes input from LiDAR data, IMU data, and robot motion control commands. These inputs construct a map of the robot's local environment and estimate its trajectory. The global SLAM algorithm is responsible for merging maps from different local SLAM sessions to create a globally consistent map. It considers the correlation between multiple local maps and the transformation of the robot's pose across different local maps. The loop closure module performs loop closure operations, such as adjusting the robot's pose or optimizing the entire map, to minimize errors caused by loops and ensure map consistency.
2. The neural network's parameters are initialized. The actor chooses the action based on the behavior policy. Random noise is appended to the action chosen by the policy network. The action a_t is subsequently transmitted to the environment for its execution.
3. The proposed BEE reward algorithm utilizes sensor data and map information from the SLAM stage. This valuable information assists the reward function in accurately describing the current state s_t of the agent (robot).
4. After the environment has executed the action, return to reward r_t and new state s_{t+1} are achieved.
5. The actor stores the state transition (s_t, a_t, r_t, s_{t+1}) in the replay memory as the training set of the online network.
6. The DDPG creates two copies of neural networks for the policy and the Q networks, respectively: the online and target networks. The update method of the policy network is as follows:

$$\begin{cases} \text{online} : Q(s, a | \theta^u), \text{ gradient update } \theta^u \\ \text{target} : Q(s, a | \theta^{u'}), \text{ soft update } \theta^{u'} \end{cases} \quad (4)$$

The Q network update method is as follows:

$$\begin{cases} \text{online} : Q(s, a | \theta^Q), \text{ gradient update } \theta^Q \\ \text{target} : Q(s, a | \theta^{Q'}), \text{ soft update } \theta^{Q'} \end{cases} \quad (5)$$

N transition data are randomly sampled from the replay memory as minibatch training data of the online policy network and online Q network. Single transition data are represented by (s_i, a_i, r_i, s_{i+1}) .

7. Critically, the Q gradient of the online Q network is calculated. The loss of the Q network is defined as follows:

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2 \quad (6)$$

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{u'}) | \theta^{Q'}) \quad (7)$$

The gradient for L and θ^Q can be obtained: $\nabla_{\theta^Q} L$, where the calculation uses the target policy network μ' and target Q network Q' .

8. The online Q is updated and θ^Q is updated by using the Adam optimizer.
9. The online policy network is updated and θ^u is updated with the Adam optimizer.
10. In the actor, the policy gradient of the policy network is calculated as follows:

$$\nabla_{\theta^u} J_{\beta}(\mu) \approx \frac{1}{k} \sum_i \nabla_a Q(s, a | \theta^Q) \big|_{s=s_i, a=\omega(s_i)} \nabla_{\theta^u} \mu(s | \theta^u) \big|_{s=s_i} \quad (8)$$

11. The parameters of the target network adopt the method of soft update:

$$\begin{cases} \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{u'} \leftarrow \tau \theta^u + (1 - \tau) \theta^{u'} \end{cases} \quad (9)$$

3.2. Proposed Exploration and Exploitation Reward Functions

The selection of the reward function is a crucial aspect of the DDPG-based Active SLAM framework. It should encompass task-specific knowledge that the agent utilizes to learn optimal actions a_t . In this case, the action of the agent includes the linear velocity v and the angular velocity ω of the robot. We proposed the following three reward functions: grid completeness, exploration, and exploitation.

3.2.1. Grid Completeness Reward

A grid completeness reward function is the first straightforward option for learning to map an unknown environment. The objective of SLAM in robotics is to construct and update a map of an unexplored environment by using equipped sensors, such as 2D LiDAR. The map constructed by the robot through LiDAR is a two-dimensional grid map represented by a grid composed of grids. The grid can store different values representing different meanings, as shown in Figure 3. White represents a free, passable area with a stored value of 0. Black represents an occupied, non-passable area with a stored value of 100. Gray represents an unknown area, where it is not explored whether the grid is passable, with a stored value of -1 . In the ROS, an occupancy map is provided that depicts the occupied and unoccupied areas of a given environment. The grid completeness M_c is naturally defined as follows:

$$M = \lambda \frac{o + uo}{X} \quad (10)$$

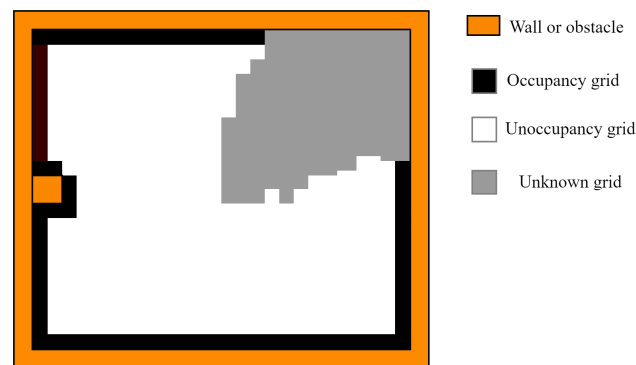


Figure 3. Example of SLAM results: occupancy grid map and composition.

The number of unoccupied cells in the occupancy grid map are represented by uo , while the number of occupied cells are represented by o . X denotes the total number of cells. Additionally, there is a scaling factor, denoted as λ , between 0 and 1. This scaling factor, λ , is determined based on the actual area of the environment to be explored. Due to small mapping errors, the map is rarely completed exactly at 100%. Therefore, we set the M_c threshold to 95% to ensure that such a condition is never not reached. The function is defined as follows:

$$r_1(s_t) = \begin{cases} r_{mapdone}, & \text{if } M \geq 95\% \\ r_{crash}, & \text{if } dis \leq L_{min} \\ M_t - M_{t-1}, & \text{otherwise} \end{cases} \quad (11)$$

where $r_{mapdone}$ is a positive scalar when the map is completed at a certain threshold, indicated by 95%, depending on a scaling factor λ . The scalar r_{crash} is negative if a terminal collision state is reached, meaning that a collision occurs. If the robot reaches a collision state, the scalar r_{crash} is negative, indicating that the distance (dis) between the robot and the obstacle is less than the minimum LiDAR detection distance L_{min} . This situation implies that a collision is imminent. Additionally, this term is proportional to the difference in grid

completeness between the current and previous time steps, where M_t is the completeness at time t and M_{t-1} is the completeness at time $t - 1$.

3.2.2. Exploration Reward

We propose a reward function based on exploration to motivate the robot to actively explore unknown environments, particularly areas where the LiDAR did not detect obstacles in the previous moment. Assuming the current time is t , we obtain the scan data o_i^{t-1} of LiDAR at the previous time $t - 1$, stored in the ROS topic: `/scan`. $o_{i,i=[0^\circ, 1^\circ, \dots, 359^\circ]}^{t-1}$ contains the distance (m) measured by the laser to obstacles at different angles within the 0° to 359° range, as shown in Figure 4a. The robot identifies the surrounding walls in the last position, P_{t-1} , and represents them on the map as a red point cloud. When an obstacle is beyond the radar's sensing range, the corresponding data in o_i^{t-1} are represented as *Inf*, such as $o_{88^\circ} = \text{Inf}$. The reward function design for exploration considers the angular range where these values are *Inf*.

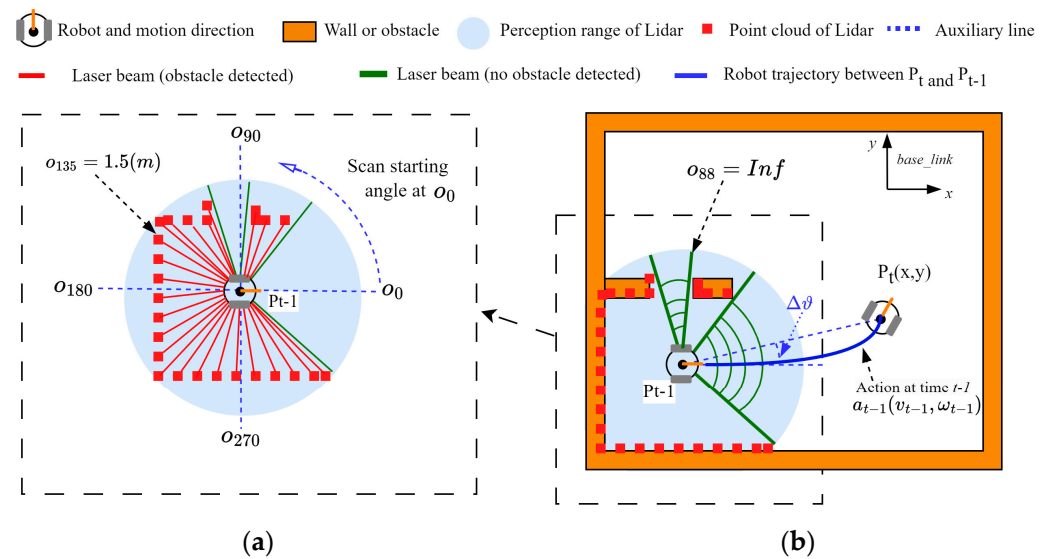


Figure 4. Example of robot LiDAR scanning. (a) Laser point cloud and angle. (b) Robot's LiDAR scanned results at the last moment and with a changed heading angle.

The exploration-driven reward function considers these angular ranges N , within which these values are *Inf*. At time $t - 1$, the robot takes action $a_{t-1}(v_{t-1}, \omega_{t-1})$ and moves to point p_t . The change in the robot's heading angle, $\Delta\theta$, can be calculated. If $(\Delta\theta + 0^\circ)$ belongs to N , a positive reward value is given; otherwise, a negative value is given. Algorithm 1 shows the proposed algorithm process. The exploration reward function is defined as follows:

$$r_2(s_t) = \begin{cases} \tau \cdot \|p_t(x, y) - p_{t-1}\|_2, & \text{if } (0^\circ + \Delta\theta) \in N \\ -\tau \cdot \|p_t(x, y) - p_{t-1}\|_2, & \text{if } (0^\circ + \Delta\theta) \notin N \end{cases} \quad (12)$$

where $\|p_t(x, y) - p_{t-1}\|_2$ is the distance from the current position p_t of the robot at time t , concerning the p_{t-1} expressed in the robot's coordinate frame. The discount factor τ adjusts the degree of reward and punishment.

Algorithm 1. The proposed exploration reward.**Input:** ROS topic: /scan; Action: $a_{t-1}(v_{t-1}, \omega_{t-1})$; Grid completeness: M_c **Output:** Reward: r_2

```

/scan  $\leftarrow$  ROS topic of robot LiDAR information
/map  $\leftarrow$  ROS topic of map information
 $o_{i,i=[0^\circ, 1^\circ, \dots, 359^\circ]}^{t-1} \leftarrow$  The list of laser scan range values (total 360 data) in the /scan at time  $t - 1$ 
 $p_t(x, y) \leftarrow$  The position of the robot at time  $t$ 
 $p_{t-1} \leftarrow$  The position of the robot at the last time,  $t - 1$ 
 $\Delta\theta \leftarrow$  Changes in the angle of heading angle when robot moved from  $p_{t-1}$  to  $p_t$ 
 $v \leftarrow$  Linear velocity of the robot
 $\omega \leftarrow$  Angular velocity of the robot
 $a_{t-1}(v_{t-1}, \omega_{t-1}) \leftarrow$  The agent took action at the last time,  $t - 1$ 
 $\tau \in (0, 1) \leftarrow$  Discount factor
1. Initialize scan memory  $L_{scan}$ 
2. Initialize the list:  $N$ 
3. while  $M_c \leq 96\%$  perform
4.   Subscribe the topic /scan
5.   Store the LiDAR scan data with  $p_t$ :  $L_{scan} \leftarrow o_i^t, p_t$ 
6.   Obtain  $o_i^{t-1}$  from  $L_{scan}$  according to  $t$ 
7.   for  $i \leftarrow 0$  to 359, perform
8.     if  $o_i^{t-1} = Inf$ , perform
9.        $N \leftarrow i$ 
10.    end
11.  end
12.  Calculate  $\Delta t \leftarrow t - (t - 1)$ 
13.  Set robot motion direction angle at  $p_{t-1}$  to  $0^\circ$ 
14.   $\Delta\theta \leftarrow \omega_{t-1} * \Delta t$ 
15.  if  $(0^\circ + \Delta\theta) \in N$ , perform
16.     $r_2 \leftarrow \tau \cdot \|p_t(x, y) - p_{t-1}\|_2$ 
17.  else  $r_2 \leftarrow -\tau \cdot \|p_t(x, y) - p_{t-1}\|_2$ 
18.  end
19. end

```

3.2.3. Exploitation Reward

Exploitation, like loop closure, is valuable as a navigating agent since it can be used for efficient exploration. The novel exploitation-driven reward function is proposed to enhance the path planning and behavioral learning process. Unlike the exploration-driven reward function, the design idea of the exploitation reward function is to try to keep the robot away from its original path point during the exploration stage and explore within a certain range. As depicted in Figure 5a, the robot (agent) continuously explores unknown areas by acquiring movement strategies, resulting in a more comprehensive occupation map. Nevertheless, it is unavoidable that the map accuracy may be compromised, leading to small, unmapped areas. Figure 5b illustrates the anticipated trajectory of the robot during the exploration process, wherein it traverses various paths to enhance map accuracy and ensure complete area detection. Algorithm 2 describes the specific calculation process. It focuses on the trajectory denoted as $Path \{p_0, p_1, \dots, p_t\}$, where p_t represents the agent's position at time t . When the robot successfully moves away from the original waypoint, the reward value will be gradually increased to reinforce this behavior positively. Conversely, the robot takes some penalty value if a previous waypoint is revisited. The exploitation reward function is defined as follows:

$$r_3(s_t) = \begin{cases} \rho \cdot \|p_t - p_{t-1}\|_2, & \text{if } \|p_t - p_i\|_2 \geq dis_{min} \\ -\rho \cdot \|p_t - p_{t-1}\|_2, & \text{if } \|p_t - p_i\|_2 \leq dis_{min} \end{cases} \quad (13)$$

where $\|p_t - p_{t-1}\|_2$ is the distance from the current position p_t of the robot at time t , concerning the p_{t-1} expressed in the robot's coordinate frame. The discount factor ρ adjusts the degree of reward and punishment. The distance threshold between two path points, denoted as dis_{min} , can be adjusted based on the actual environmental conditions.

Algorithm 2. The proposed exploitation reward.

Input: ROS topic: /map, /pose; Action: $a_{t-1}(v_{t-1}, \omega_{t-1})$; Grid completeness: M_c

Output: Reward: r_3

```

/map ← ROS topic of map information
/pose ← ROS topic of robot position information
 $l_t \leftarrow$  Loop closure label
 $dis_{min} \leftarrow$  Minimum distance values
 $p_t \leftarrow$  The position of the robot at time  $t$ 
 $v \leftarrow$  Linear velocity of the robot
 $\omega \leftarrow$  Angular velocity of the robot
 $a_{t-1}(v_{t-1}, \omega_{t-1}) \leftarrow$  The agent took action at the last time,  $t - 1$ 
 $\rho \in (0, 1) \leftarrow$  Discount factor
1. Initialize path memory  $Path \{p_0, p_1, \dots\}$ 
2. Subscribe the topic: /pose
3. Store the position data:  $Path \{\} \leftarrow p_t$ 
4. while  $M_c \geq 96\%$  perform
5.   for  $i$  in  $Path$  perform
6.     if  $\|p_t - p_i\|_2 \geq dis_{min}$ , perform
7.        $r_3 \leftarrow \rho \cdot \|p_t(x, y) - p_{t-1}\|_2$ 
8.     else  $r_3 \leftarrow -\rho \cdot \|p_t(x, y) - p_{t-1}\|_2$ 
9.     end
10.  end
11. end

```

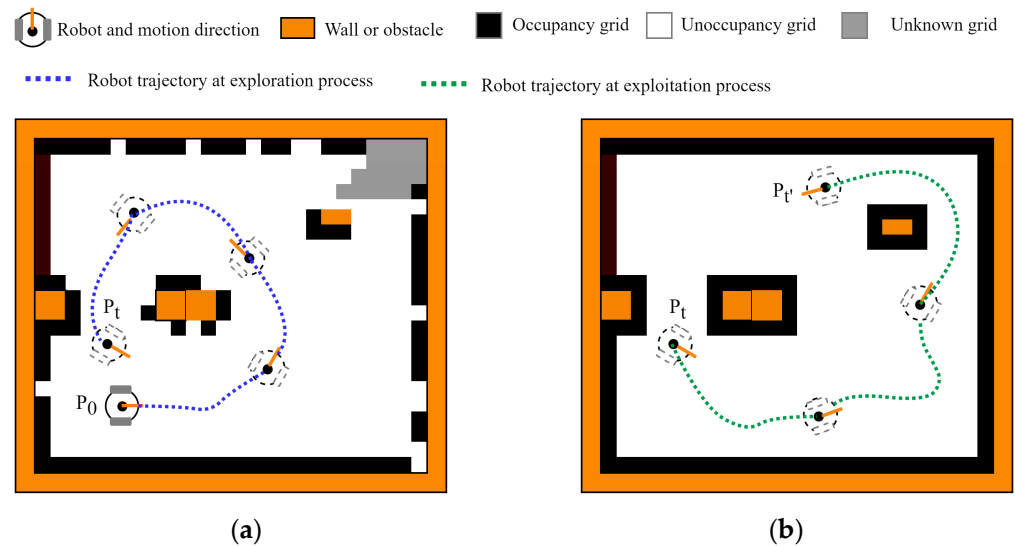


Figure 5. Example of different trajectories during robot exploration and exploitation process. (a) Robot exploration trajectory. (b) Robot exploitation trajectory.

In addition, the reward function of the agent's state s_t at t is defined as follows:

$$R(s_t) = \alpha r_1(s_t) + \beta r_2(s_t) + \gamma r_3(s_t) \quad (14)$$

where α , β , and γ are scalar weighting factors for the three reward terms $r_1(s_t)$, $r_2(s_t)$, and $r_3(s_t)$.

3.3. State and Action Spaces

The DDPG is chosen as the path planning algorithm for determining the robot's velocity commands (linear and angular velocities) to navigate to target locations without colliding with obstacles. In particular, the linear velocity is a continuous function limited in the range $[0, 1]$ to allow for only forward motion (no backward), and the angular velocity is a constant function specified in the range $[-1, 1]$ to allow for right and left rotations. The state vector s_t is composed of 180 LiDAR data points of the 360-degree range o , the action chosen at the previous time step a_{t-1} , and the current occupied map m . The state vector is defined as shown in Equation (20).

$$s_t = [o, a_{t-1}, m] \quad (15)$$

4. Simulation Results

4.1. Simulation Setup

The experiments are performed in a 3D simulation environment, using the Robot Operating System (ROS) 3D visualizer (Rviz) [33] and the Gazebo simulator [34], with a differential drive mobile robot (TurtleBot 3) [35] equipped with 360° laser range scanner (LiDAR) and wheel odometry. The LiDAR range is between 0.12 m and 3.5 m. The experiments are conducted on an Ubuntu 20.0 machine with ROS Noetic. Additionally, the ROS allows for easy integration with different SLAM packages and the Cartographer package [12] that is used in this research. The reinforcement learning algorithm is written in Python 3.8 using the TensorFlow 2.2 library to construct and train the neural networks and OpenAI Gym for the reinforcement learning environment.

4.1.1. Environment Setup

In order to evaluate the proposed DDPP-based Active SLAM scheme, several environments were simulated in Gazebo, as shown in Figure 6. The TurtleBot3 starts at its initial point before taking any action. The blue line segment represents the scanning range of the robot's LiDAR. Figure 6a–c illustrate different indoor environments: Env-1 (length: 5.5 m; width: 4 m), Env-2 (length: 10.5 m; width: 7 m), and Env-3 (length: 15 m; width: 9 m). Among these environments, the one shown in Figure 6c is the most complex as it consists of multiple rooms, which challenges the robot's path planning ability. Figure 6d–f depict the preliminary SLAM map created by the robot prior to its movement.

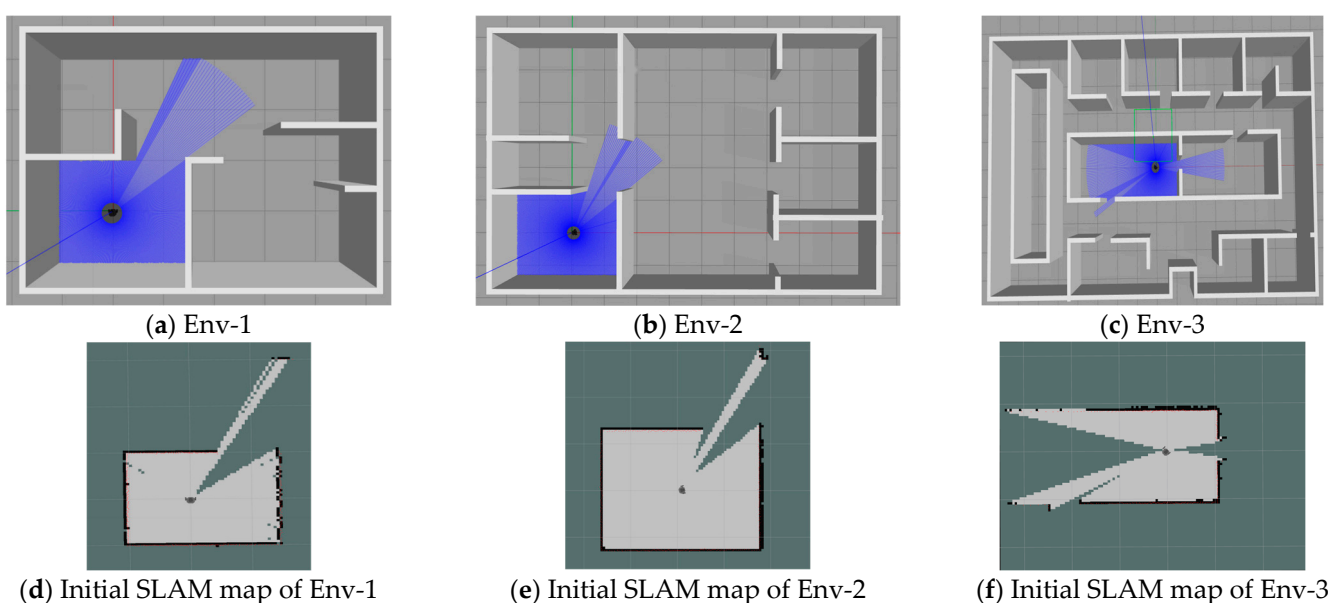


Figure 6. The environment is simulated in Gazebo, and the initial map is produced in Rviz. A TurtleBot3 robot is used as the platform.

4.1.2. Decision Making Module

Decision making was performed via DRL using the Deep Deterministic Policy Gradient (DDPG). The DDPG can handle the robot's continuous movement and action space and is more suitable for exploring Active SLAM. During the model's training, we implemented a stochastic policy gradient with the Adam optimizer [36] to train the critic and actor networks. The actor network used a learning rate of 10^{-3} , while the critic network was updated using a learning rate of 10^{-4} . To prevent overfitting, we included L2 regularization with a coefficient of 10^{-2} when training the critic network. Moreover, we employed a discount factor of $\gamma = 0.99$ and set the target update rate to $\tau = 0.001$. To incorporate random noise, we utilized the Ornstein–Uhlenbeck process with parameter σ . $B_{size} = 64$ is the number of samples obtained in one training stage. During a training/testing stage, the robot passes through several episodes. In each episode, the robot moves until it collides with an obstacle (closer than a defined threshold of 0.12 m) or consumes 1000 steps (C). The agents were trained in Env-2, which has a height of 10.5 m and a width of 7.0 m. The training involved a total of 6×10^6 time steps (T) and a maximum of 2000 steps per episode. The complete list of parameter values used in the DDPG can be found in Table 2.

Table 2. Hyperparameters of the DDPG.

Parameter	Description	Value
Opt	Adam optimizer	ADAM
Lr_{actor}	Learning rate for actor network	10^{-3}
Lr_{critic}	Learning rate for critic network	10^{-4}
$L2_{re}$	L2 regularization coefficient	10^{-2}
γ	Discount factor	0.99
τ	Target network update	0.001
σ	OU random noise	0.15
B_{size}	Batch size	64
T	Total time steps	6×10^6
C	Collision threshold (m)	1000
$Step_{max}$	Maximum steps per episode	2000

4.2. Evaluation Index

Evaluation index for SLAM map: A comparative analysis of the accuracy and quality of maps generated by SLAM algorithms was conducted using full reference image quality assessment (FR-IQA). The reference image (ref) is the map created when we manually control the robot's movement. The robot is manually controlled to navigate different environmental areas at a low and constant speed, ensuring repeated traversals to enhance the mapping accuracy through closed-loop detection of the SLAM algorithm. The FR-IQA methods used were Mean Square Error (MSE) [37], Peak Signal-to-Noise Ratio (PSNR) [38], and the Structural Similarity Index Measure (SSIM) [39]. The SSIM is a metric commonly used to compare two maps of the same size, for example, in terms of length and width. It calculates the mean, variance, and covariance of cell intensities from a given image. The SSIM values range from -1 to $+1$, where a value of $+1$ indicates that the two maps are identical.

Evaluation index for path planning: The ability to plan a path is crucial in evaluating Active SLAM. One fundamental metric is path length, with shorter paths being preferable. Another critical factor is running time, especially in quick decision making scenarios. Additionally, it is important to consider the training efficiency of reinforcement learning algorithms. By comprehensively evaluating these indicators, we can better understand the performance of the path planning algorithm in different application scenarios. We used the Evo toolbox to provide detailed comparisons to visualize the robot's trajectory in various scenarios. We design the cost function to consider both the energy consumption of the

robot when it moves (measured in Joules) and the area of the environment that the robot effectively maps (measured in square meters). The cost function is defined as follows:

$$Cost = \frac{E}{A} \left(J/m^2 \right) \quad (16)$$

$$A = \text{Grid Completeness}(\%) \cdot \text{The total area} (m^2) \quad (17)$$

$$E = 0.5 \cdot \text{path length}(m) \cdot 1J(J/m) + 0.5 \cdot \text{running time}(s) \cdot 2J(J/s) \quad (18)$$

where the area (A) of the successfully mapped environment is determined by the total area of the simulated scenario and the actual grid completeness. Energy consumption (E) is divided into two parts. The robot consumes 1 J for every 1 m it moves and 2 J for every 1 s it moves.

4.3. Evaluation Results

We compared the proposed approach to two different Active SLAM methods, which use frontier detection-based exploration (FD) [19] and D3Qn- and D-opt-based SLAM (DDS) [29] in three simulation environments: Env-1, Env-2, and Env-3.

4.3.1. Comparison of Training Results

To compare the training performance of RL agents, we analyzed the collision rate concerning the number of episodes. The calculation formula of collision rate is as follows:

$$\text{Collision rate} = \frac{N_c}{T_E} \times 100\% \quad (19)$$

where N_c represents the number of collisions that occurred during training. T_E represents the total number of episodes performed during training. The formula gives a percentage representing the average collision rate per episode during training. The training results of the DDS algorithm and our proposed algorithm, which both utilize reinforcement learning neural networks, were compared. The results are presented in Figure 7.

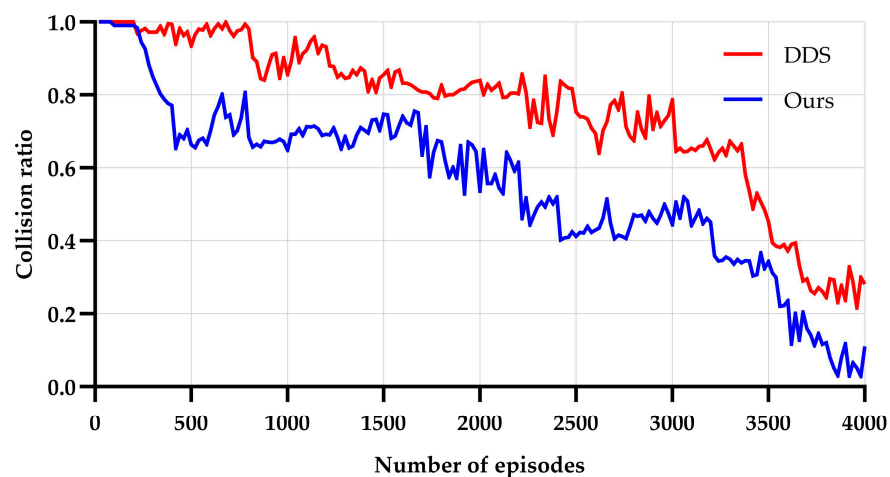


Figure 7. Evolution of the collision ratio with the number of training episodes.

In Figure 7, the red line represents an agent trained with the DDS algorithm. This agent aims to minimize collisions during training, as it is penalized after collisions have occurred. As a result, its collision rate gradually decreases to 0.2 after 4000 training episodes. On the other hand, the agent trained using our proposed algorithm consistently achieves a collision rate below 0.2 within 3600 training episodes, indicating superior training performance. Our reward function emphasizes exploration and exploitation, motivating the agent to

explore the environment, navigate open areas, and avoid getting stuck in local minima. Furthermore, agents trained using our proposed method demonstrate higher robustness and generalization capabilities throughout the training process, as evidenced by their ability to reduce collision rate fluctuations consistently.

4.3.2. Comparison of SLAM Results

We applied four methods for the SLAM mapping task in three scenarios: Env-1, Env-2, and Env-3. The first method involved manually controlling the robot to explore all unknown areas without considering the time cost, as shown in Figure 8a. The SLAM image obtained through manual control exhibited clearer and more accurate edges. The remaining three methods were Active SLAM methods. The FD method guided the robot to explore the edges of both known and unknown areas. However, in complex scenarios like Env-3, the particle-based SLAM method led to increased computational load and decreased map accuracy, as confirmed in Table 3. DDS and our proposed method utilized reinforcement learning models to train robots (agents) to find optimal paths and employed SLAM algorithms for autonomous mapping. In Env-2 and Env-3, the graph generated by the DDS algorithm was not fully complete, indicating that the robot did not effectively explore all unknown areas.



Figure 8. SLAM maps were built for three environments by manually controlling and using different Active SLAM methods using Rviz software. (a) Manually controlled (MC). (b) Frontier detection-based exploration (FD). (c) D3Qn- and D-opt-based SLAM (DDS). (d) The proposed method.

In contrast, our proposed method generated graphs that resembled those produced through manual control. The reference image was also generated under manual control, while the evaluated images were generated using various algorithms. The specific values are presented in Table 3.

Our proposed algorithm achieved the best MSE value of 0.3 through comparison with other methods in the Env-1 environment. In the more complex Env-3 environment, it also exhibited the highest SSIM value of 0.64, indicating a relatively high accuracy for the SLAM results.

Table 3. The quality comparison of SLAM map in three environments. The reference image is generated by manually controlling the robot.

Scenarios	Methods	MSE	PSNR (dB)	SSIM
Env-1	MC ¹	0	Inf	1
	FD ²	0.55	25	0.63
	DDS ³	0.51	26	0.72
	Ours	0.30	37	0.88
Env-2	MC ¹	0	Inf	1
	FD ²	0.61	20	0.59
	DDS ³	0.59	27	0.61
	Ours	0.41	35	0.74
Env-3	MC ¹	0	Inf	1
	FD ²	0.81	25	0.43
	DDS ³	0.71	21	0.37
	Ours	0.54	30	0.64

¹ Manually controlled. ² Frontier detection-based exploration [19]. ³ D3Qn- and D-opt-based SLAM [29].

4.3.3. Comparison of Robot Trajectories

Agents (robots) were evaluated in three environments: Env-1, Env-2, and Env-3. The results of the robot operating in these environments are presented in Table 4. The mappings in Env-2 and their respective generated paths are illustrated in Figure 4, while Figure 9 shows the mappings in Env-1 and their respective generated paths. Additionally, Figure 10 displays the mappings in Env-2 and their respective generated paths. In each map, the blue line segment represents the actual movement trajectory of the robot. It is important to note that our proposed algorithm consists of two key stages: exploration and exploitation. During the exploration phase, the robot extensively traverses unknown areas to increase the rate of map exploration. In the exploitation phase, the robot focuses on untraversed areas to improve grid completeness further. Firstly, our exploration phase outperforms path length, time, and grid completion. In Env-1 and Env-2, compared to the MC, FD, and DDS methods, the average path length of our exploration phase is reduced by 15.5% and 23.4%, respectively, and the time is reduced by 20.6% and 13.9%. Moreover, the grid completion levels increased by 0.8% and 1.9%, respectively.

Table 4. Quality comparison of the SLAM map.

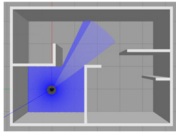
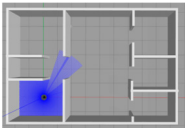
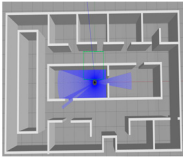
Scenarios	Methods	Path Length (m)	Time (s)	Grid Completeness (%)	Cost Value (J/m ²)
Env-1 	MC ¹	25.02	230	99.2	22.2
	FD ²	22.1	215	97.9	20.9
	DDS ³	23.3	195	97.7	19.2
	Ours ⁴	Explor ⁵	184	98.0	18.1
		Exploit ⁶	254	98.7	-
Env-2 	MC ¹	36.8	378	98.7	10.9
	FD ²	39.7	346	95.4	10.4
	DDS ³	42.6	354	95.1	10.7
	Ours ⁴	Explor ⁵	332	96.1	9.9
		Exploit ⁶	216	98.2	-

Table 4. Cont.

Scenarios	Methods	Path Length (m)	Time (s)	Grid Completeness (%)	Cost Value (J/m ²)
Env-3 	MC ¹	97.8	1489	97.9	23.2
	FD ²	123.9	1332	91.6	22.5
	DDS ³	100.5	965	84.2	17.8
	Ours ⁴ Explor ⁵	111.8	924	95.3	15.2
	Exploit ⁶	134.6	634	96.1	-

¹ Manually controlled. ² Frontier detection-based exploration [19]. ³ D3Qn- and D-opt-based SLAM [29]. ⁴ Our proposed approach, which focuses on two phases: exploration and exploitation. ⁵ Exploration process. ⁶ Exploitation process.

Secondly, our exploitation phase significantly enhances grid completeness. Compared to the exploitation stage of other algorithms, our method achieves average improvements of 2.5%, 3.7%, and 6.5% in grid completeness for Env-1, Env-2, and Env-3, respectively. Furthermore, when the map is larger and more complex, such as in Env-3, as shown in Figure 11, our proposed method enables the robot to cover a distance of only 111.8 m during the exploration phase, achieving a grid completeness that is 11.1% higher than that of the DDS method. Grid completeness was further improved during the exploitation phase. The cost value in Table 4 compares the energy consumption of a robot exploring unknown areas using various algorithms. The results show that our proposed algorithm effectively reduces the energy consumption of the robot. For example, the robot only consumes 15.2 J of energy per square meter of the explored area in Env-3. In Env-1 and Env-2, energy consumption is also notably lower compared to other algorithms.

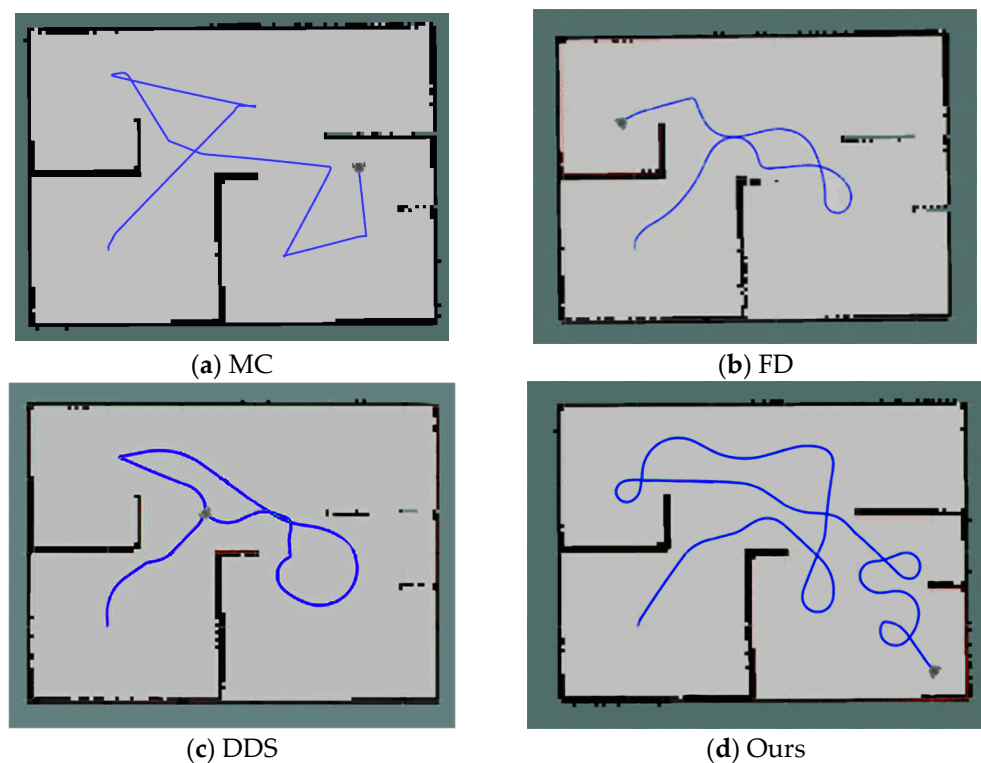


Figure 9. Comparison of movement trajectories (blue line) during robot mapping for different methods in Env-1 using Rviz software. (a) Manually controlled (MC). (b) Frontier detection-based exploration (FD). (c) D3Qn- and D-opt-based SLAM (DDS). (d) The proposed method.

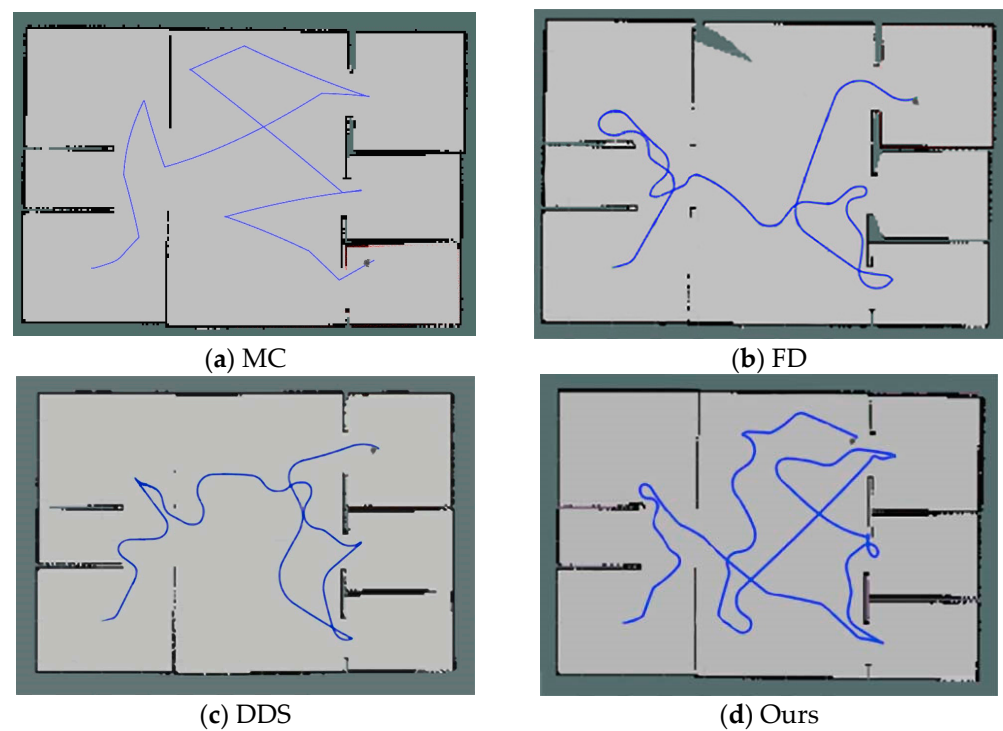


Figure 10. Comparison of movement trajectories (blue line) during robot mapping for different methods in Env-2 using Rviz software. (a) Manually controlled (MC). (b) Frontier detection-based exploration (FD). (c) D3Qn- and D-opt-based SLAM (DDS). (d) The proposed method.

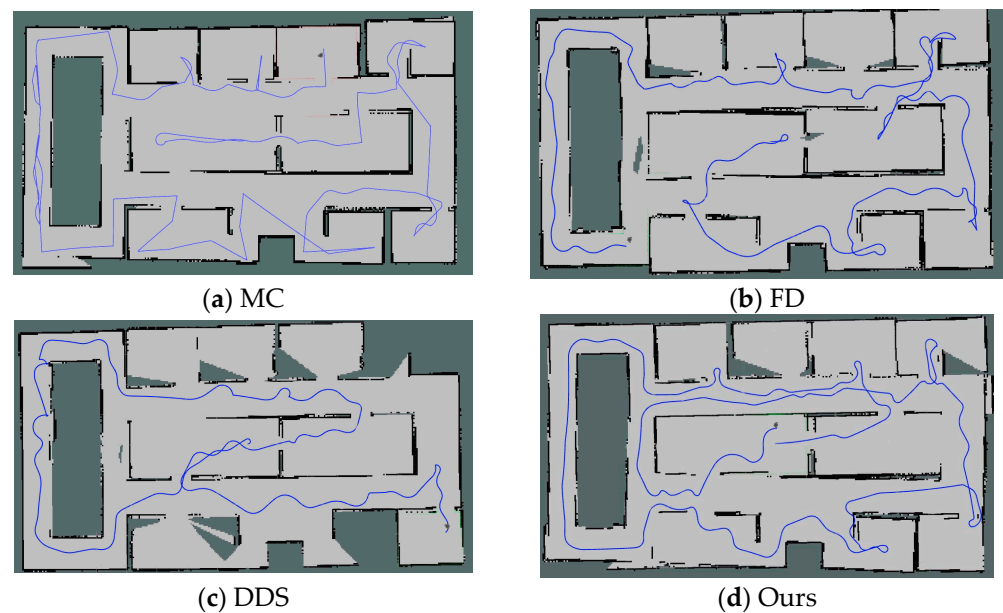


Figure 11. Comparison of movement trajectories (blue line) during robot mapping for different methods in Env-3 using Rviz software. (a) Manually controlled (MC). (b) Frontier detection-based exploration (FD). (c) D3Qn- and D-opt-based SLAM (DDS). (d) The proposed method.

5. Conclusions

This study proposes a novel solution for Active Simultaneous Localization and Mapping (Active SLAM) that allows robots to effectively navigate and map unknown indoor environments. This solution utilizes a combination of Deep Deterministic Policy Gradient (DDPG) path planning and the Cartographer algorithm. The key to this method is a unique reward function, which integrates exploration and exploitation strategies to accelerate

network convergence and minimize collisions with obstacles. The experimental results demonstrate that this innovative methodology surpasses other Active SLAM methods in exploring and mapping unknown environments, achieving an impressive grid completion rate of 98.7%. These outcomes validate the effectiveness of the novel approach, highlighting its advantages in improving map accuracy and traversal efficiency. In future work, we will continue to improve the reinforcement learning network strategy and apply the mapping results to autonomous navigation.

Author Contributions: S.Z. and S.-H.H. contributed to the main idea of this research work. S.Z. wrote the computation codes and performed the simulations, experiments, and database collection. S.Z. and S.-H.H. confirmed the numerical results of the work. The research activity was planned and executed under the supervision of S.-H.H. In addition, S.Z. and S.-H.H. contributed to the writing of this article. All authors have read and agreed to the published version of the manuscript.

Funding: The following are the results of a study on the “Leaders in Industry-university Cooperation3.0” Project supported by the Ministry of Education and National Research Foundation of Korea.

Data Availability Statement: Data is contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Filip, I.; Pyo, J.; Lee, M.; Joe, H. LiDAR SLAM with a Wheel Encoder in a Featureless Tunnel Environment. *Electronics* **2023**, *12*, 1002. [CrossRef]
2. Zhao, S.; Hwang, S.-H. ROS-Based Autonomous Navigation Robot Platform with Stepping Motor. *Sensors* **2023**, *23*, 3648. [CrossRef] [PubMed]
3. Zhao, S.; Hwang, S.H. Complete coverage path planning scheme for autonomous navigation ROS-based robots. *ICT Express* **2024**, *10*, 83–89. [CrossRef]
4. Guo, F.; Yang, H.; Wu, X.; Dong, H.; Wu, Q.; Li, Z. Model-Based Deep Learning for Low-Cost IMU Dead Reckoning of Wheeled Mobile Robot. *IEEE Trans. Ind. Electron.* **2023**, 1–11. [CrossRef]
5. Motta, J.M.S.T.; de Carvalho, G.C.; McMaster, R.S. Robot Calibration Using a 3D Vision-Based Measurement System with a Single Camera. *Robot. Comput.-Integr. Manuf.* **2001**, *17*, 487–497. [CrossRef]
6. Bailey, T.; Nieto, J.; Guivant, J.; Stevens, M.; Nebot, E. Consistency of the EKF-SLAM Algorithm. In Proceedings of the 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems, Beijing, China, 9–15 October 2006; pp. 3562–3568.
7. Nieto, J.; Bailey, T.; Nebot, E. Recursive Scan-Matching SLAM. *Robot. Auton. Syst.* **2007**, *55*, 39–49. [CrossRef]
8. Zhang, Q.; Wang, P.; Chen, Z. An Improved Particle Filter for Mobile Robot Localization Based on Particle Swarm Optimization. *Expert Syst. Appl.* **2019**, *135*, 181–193. [CrossRef]
9. Montemerlo, M.; Thrun, S. Simultaneous Localization and Mapping with Unknown Data Association Using FastSLAM. In Proceedings of the 2003 IEEE International Conference on Robotics and Automation (Cat. No.03CH37422), Taipei, Taiwan, 14–19 September 2003; Volume 2, pp. 1985–1991.
10. Grisetti, G.; Stachniss, C.; Burgard, W. Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters. *IEEE Trans. Robot.* **2007**, *23*, 34–46. [CrossRef]
11. Arshad, S.; Kim, G.-W. Role of Deep Learning in Loop Closure Detection for Visual and Lidar SLAM: A Survey. *Sensors* **2021**, *21*, 1243. [CrossRef]
12. Cartographer ROS Integration—Cartographer ROS Documentation. Available online: <https://google-cartographer-ros.readthedocs.io/en/latest/> (accessed on 28 December 2023).
13. Ahmed, M.F.; Masood, K.; Fremont, V.; Fantoni, I. Active SLAM: A Review on Last Decade. *Sensors* **2023**, *23*, 8097. [CrossRef]
14. Yamauchi, B. A Frontier-Based Approach for Autonomous Exploration. In Proceedings of the 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. “Towards New Computational Principles for Robotics and Automation”, Monterey, CA, USA, 10–11 July 1997; pp. 146–151.
15. Li, B.; Chen, B. An adaptive rapidly-exploring random tree. *IEEE/CAA J. Autom. Sin.* **2021**, *9*, 283–294. [CrossRef]
16. Placed, J.A.; Castellanos, J.A. A general relationship between optimality criteria and connectivity indices for active graph-SLAM. *IEEE Robot. Autom. Lett.* **2022**, *8*, 816–823. [CrossRef]
17. Gul, F.; Mir, I.; Abualigah, L.; Sumari, P.; Forestiero, A. A Consolidated Review of Path Planning and Optimization Techniques: Technical Perspectives and Future Directions. *Electronics* **2021**, *10*, 2250. [CrossRef]
18. Wang, H.; Yu, Y.; Yuan, Q. Application of Dijkstra Algorithm in Robot Path-Planning. In Proceedings of the 2011 Second International Conference on Mechanic Automation and Control Engineering, Hohhot, China, 15–17 July 2011; pp. 1067–1069.
19. Carlone, L.; Du, J.; Kaouk Ng, M.; Bona, B.; Indri, M. Active SLAM and Exploration with Particle Filters Using Kullback-Leibler Divergence. *J. Intell. Robot. Syst.* **2014**, *75*, 291–311. [CrossRef]

20. Trivun, D.; Šalaka, E.; Osmanković, D.; Velagić, J.; Osmić, N. Active SLAM-Based Algorithm for Autonomous Exploration with Mobile Robot. In Proceedings of the 2015 IEEE International Conference on Industrial Technology (ICIT), Seville, Spain, 17–19 March 2015; pp. 74–79.
21. Mihálik, M.; Malobický, B.; Peniak, P.; Vestenický, P. The New Method of Active SLAM for Mapping Using LiDAR. *Electronics* **2022**, *11*, 1082. [CrossRef]
22. Placed, J.A.; Castellanos, J.A. Fast Autonomous Robotic Exploration Using the Underlying Graph Structure. In Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 27 September–1 October 2021; pp. 6672–6679.
23. Suresh, S.; Sodhi, P.; Mangelson, J.G.; Wettergreen, D.; Kaess, M. Active SLAM Using 3D Submap Saliency for Underwater Volumetric Exploration. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020; pp. 3132–3138.
24. Hester, T.; Vecerik, M.; Pietquin, O.; Lanctot, M.; Schaul, T.; Piot, B.; Horgan, D.; Quan, J.; Sendonaris, A.; Osband, I.; et al. Deep Q-Learning From Demonstrations. In Proceedings of the AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018; Volume 32.
25. Li, X.; Chen, G.; Wu, G.; Sun, Z.; Chen, G. Research on Multi-Agent D2D Communication Resource Allocation Algorithm Based on A2C. *Electronics* **2023**, *12*, 360. [CrossRef]
26. Cimurs, R.; Lee, J.H.; Suh, I.H. Goal-Oriented Obstacle Avoidance with Deep Reinforcement Learning in Continuous Action Space. *Electronics* **2020**, *9*, 411. [CrossRef]
27. Chen, F.; Martin, J.D.; Huang, Y.; Wang, J.; Englot, B. Autonomous Exploration Under Uncertainty via Deep Reinforcement Learning on Graphs. In Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 24 October 2020–24 January 2021; pp. 6140–6147.
28. Placed, J.A.; Rodríguez, J.J.G.; Tardós, J.D.; Castellanos, J.A. ExplORB-SLAM: Active Visual SLAM Exploiting the Pose-Graph Topology. In *Proceedings of the ROBOT2022: Fifth Iberian Robotics Conference*; Tardioli, D., Matellán, V., Heredia, G., Silva, M.F., Marques, L., Eds.; Springer International Publishing: Cham, Switzerland, 2023; pp. 199–210.
29. Placed, J.A.; Castellanos, J.A. A Deep Reinforcement Learning Approach for Active SLAM. *Appl. Sci.* **2020**, *10*, 8386. [CrossRef]
30. Li, S.; Xu, X.; Zuo, L. Dynamic Path Planning of a Mobile Robot with Improved Q-Learning Algorithm. In Proceedings of the 2015 IEEE International Conference on Information and Automation, Lijiang, China, 8–10 August 2015; pp. 409–414.
31. Vithayathil Varghese, N.; Mahmoud, Q.H. A Survey of Multi-Task Deep Reinforcement Learning. *Electronics* **2020**, *9*, 1363. [CrossRef]
32. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous control with deep reinforcement learning. *arXiv* **2015**, arXiv:1509.02971.
33. Rviz—ROS Wiki. Available online: <https://wiki.ros.org/rviz> (accessed on 15 January 2024).
34. Gazebo. Available online: <https://gazebo.org/home> (accessed on 15 January 2024).
35. Name, Y. ROBOTIS E-Manual. Available online: <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/> (accessed on 15 January 2024).
36. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. *arXiv* **2014**, arXiv:1412.6980.
37. Kümmerle, R.; Steder, B.; Dornhege, C.; Ruhnke, M.; Grisetti, G.; Stachniss, C.; Kleiner, A. On Measuring the Accuracy of SLAM Algorithms. *Auton. Robot.* **2009**, *27*, 387–407. [CrossRef]
38. Cao, L.; Ling, J.; Xiao, X. Study on the Influence of Image Noise on Monocular Feature-Based Visual SLAM Based on FFDNet. *Sensors* **2020**, *20*, 4922. [CrossRef] [PubMed]
39. Sankalprajan, P.; Sharma, T.; Perur, H.D.; Sekhar Pagala, P. Comparative Analysis of ROS Based 2D and 3D SLAM Algorithms for Autonomous Ground Vehicles. In Proceedings of the 2020 International Conference for Emerging Technology (INCET), Belgaum, India, 5–7 June 2020; pp. 1–6.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.